
Security Review Report

NM-0861-Hyperlend



NETHERMIND
SECURITY

(April 1, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Main Functionalities	4
4.2	Trust and Access Control Model	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Medium] The repayWithTokens() differs from the Pool's implementation	6
6.2	[Low] Incorrect check order in the repayWithPermit(...) function	6
6.3	[Info] Inaccurate NatSpec references regarding credit delegation and onBehalfOf	6
6.4	[Info] Missing token rescue functionality for tokens stuck in the gateway	6
6.5	[Info] POOL state variable has no public getter, reducing on-chain transparency	7
6.6	[Info] POOL variable not set during initialization, creating non-functional deployment window	7
6.7	[Info] The borrow(...) function always requires prior credit delegation to the gateway contract	7
6.8	[Info] The repayWithPermit(...) function hardcodes the interest rate mode	8
6.9	[Info] The gateway does not wrap setUserUseReserveAsCollateral(...) and setUserEMode(...)	8
6.10	[Best Practice] The number of gap slots should sum up with other variables to a round number	8
7	Documentation Evaluation	9
8	Test Suite Evaluation	10
8.1	Tests Output	10
8.2	Automated Tools	10
8.2.1	AuditAgent	10
9	About Nethermind	11

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [HyperLend's](#) WhitelistGateway contract.

HyperLend is a permissioned lending protocol deployed on HyperEVM that layers KYC-based access control on top of an Aave v3-compatible lending pool. Users interact with the pool exclusively through WhitelistGateway, a UUPS-upgradeable proxy contract that gates all core lending operations: supply, borrow, repay, withdraw, liquidation, and flash loans behind an owner-managed whitelist. Rather than allowing whitelisted users to call the pool directly, the gateway acts as an intermediary using a pull-transfer-forward pattern: it pulls tokens from the caller via `safeTransferFrom`, grants allowances to the pool via `safeIncreaseAllowance`, and forwards the corresponding call on the user's behalf.

The gateway supports EIP-2612 permit-based flows for `supplyWithPermit` and `repayWithPermit`, wrapping the permit call in a try/catch that falls back to checking existing allowances. Delegated credit operations are explicitly blocked: both `borrow` and `flashLoan` enforce `msg.sender == onBehalfOf`. The liquidation flow uses a balance-difference pattern, snapshotting the gateway's collateral balance before the pool call and transferring only the delta to the liquidator. A separate `repayWithATokens` function converts `aTokens` to underlying via a sequential `POOL.withdraw` then `POOL.repay` call.

Governance is minimal: the contract owner (set during initialization via OpenZeppelin's `OwnableUpgradeable`) controls whitelist membership, the pool address (mutable via `setPool` with no timelock), and upgrade authorization. The contract holds no persistent token balances by design, as tokens flow through it within a single transaction.

The audit comprises 160 nSLOC of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report ten points of attention, where one is classified as Medium, one is classified as Low and eight are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

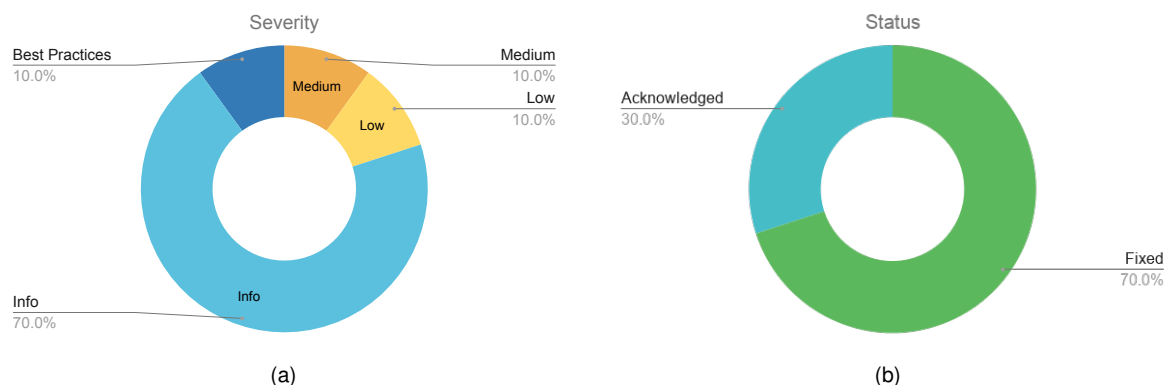


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (1), Low (1), Undetermined (0), Informational (7), Best Practices (1). Distribution of status: Fixed (7), Acknowledged (3), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	March 26, 2026
Final Report	April 1, 2026
Initial Commit	87d4f1
Final Commit	57fa601
Documentation Assessment	Medium
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/WhitelistGateway.sol	160	121	75.6%	36	317
	Total	160	121	75.6%	36	317

3 Summary of Issues

	Finding	Severity	Update
1	The repayWithTokens() differs from the Pool's implementation	Medium	Acknowledged
2	Incorrect check order in the repayWithPermit(...) function	Low	Fixed
3	Inaccurate NatSpec references regarding credit delegation and onBehalfOf	Info	Fixed
4	Missing token rescue functionality for tokens stuck in the gateway	Info	Fixed
5	POOL state variable has no public getter, reducing on-chain transparency	Info	Fixed
6	POOL variable not set during initialization, creating non-functional deployment window	Info	Acknowledged
7	The borrow(...) function always requires prior credit delegation to the gateway contract	Info	Fixed
8	The repayWithPermit(...) function hardcodes the interest rate mode	Info	Fixed
9	The gateway does not wrap setUserUseReserveAsCollateral(...) and setUserEMode(...)	Info	Acknowledged
10	The number of gap slots should sum up with other variables to a round number	Best Practices	Fixed

4 System Overview

HyperLend is a permissioned lending protocol deployed on HyperEVM that wraps an Aave v3-compatible lending pool behind a KYC-gated access layer. The sole in-scope contract, `WhitelistGateway`, is deployed as a UUPS-upgradeable proxy using OpenZeppelin contracts with a storage gap reserved for future upgrades.

The protocol provides whitelisted users with the full suite of Aave v3 lending operations: supply, borrow, repay, withdraw, liquidation, and flash loans, all routed through the gateway rather than the pool directly. EIP-2612 permit-based flows are supported for supply and repay operations, removing the need for separate approval transactions.

Governance is minimal: a single owner address (set during initialization via OpenZeppelin's `OwnableUpgradeable`) controls whitelist membership, pool configuration, and upgrade authorization.

4.1 Main Functionalities

- **Supply and Deposit:** Whitelisted users supply underlying assets (e.g. USDC) into the lending pool via the gateway, receiving `aTokens` in return. The gateway pulls tokens from the caller via `safeTransferFrom`, grants an allowance to the pool via `safeIncreaseAllowance`, and calls `POOL.supply`. A deprecated `deposit` function is maintained for backwards compatibility with identical logic. Both `supply` and `deposit` allow specifying a different `onBehalfOf` address, enabling supply on behalf of another user.
- **Borrowing:** Whitelisted users borrow underlying assets against their supplied collateral. The gateway calls `POOL.borrow` and forwards the borrowed tokens to `msg.sender`. Delegated credit operations are explicitly blocked: `msg.sender` must equal `onBehalfOf`, preventing abuse of credit delegation through the gateway.
- **Repayment:** Users repay outstanding debt through three paths: (1) `repay` transfers underlying tokens from the caller and forwards them to the pool, with the amount silently capped to the outstanding debt; (2) `repayWithATokens` converts `aTokens` to underlying via a sequential `POOL.withdraw` then `POOL.repay`, with the amount capped to the lesser of the caller's `aToken` balance and their outstanding debt; (3) `repayWithPermit` combines repayment with an EIP-2612 permit signature to avoid a prior approve transaction.
- **Withdrawal:** Users withdraw underlying assets by passing `aTokens` through the gateway. The gateway pulls `aTokens` from the caller via `safeTransferFrom` and calls `POOL.withdraw`, which burns the `aTokens` and sends the underlying to the specified address. Passing `type(uint256).max` as the amount withdraws the caller's full `aToken` balance.
- **Liquidation:** Whitelisted liquidators can liquidate undercollateralized positions (Health Factor below 1). The gateway uses a balance-difference pattern: it snapshots its own collateral balance before calling `POOL.liquidationCall`, then transfers only the delta to the liquidator. Any unused debt tokens are returned to the caller. Liquidators may choose to receive either the underlying collateral asset or the corresponding `aTokens`. Self-liquidation is explicitly blocked.
- **Flash Loans:** The gateway exposes both `flashLoan` (multi-asset) and `flashLoanSimple` (single-asset) from the underlying pool. The multi-asset variant enforces `msg.sender == onBehalfOf` to prevent delegated debt opening. The simple variant does not enforce this constraint, as it does not support opening debt positions.
- **Permit-based Operations:** `supplyWithPermit` and `repayWithPermit` wrap the EIP-2612 permit call in a try/catch block. If the permit fails (e.g. already consumed or front-run), the gateway falls back to checking the caller's existing allowance, reverting only if the allowance is insufficient.

4.2 Trust and Access Control Model

- **Single-owner governance:** The contract uses OpenZeppelin's `OwnableUpgradeable` with a single owner address controlling all privileged operations: whitelist management, pool address configuration, and UUPS upgrade authorization. There is no timelock, multisig, or role-based access control — all admin power is concentrated in one address.
- **Whitelist-gated access:** Every user-facing function is protected by the `onlyWhitelisted` modifier, which checks a `mapping(address => bool)`. Only the owner can add or remove addresses. This provides a KYC/permissioning layer without modifying the underlying Aave v3 pool contracts.
- **Mutable pool address:** The owner can change the pool address at any time via `setPool` with no timelock or two-step process. The only validation is a zero-address check. A pool change would redirect all subsequent user operations to the new pool.
- **Upgrade authority:** UUPS upgrade authorization is restricted to the owner via `_authorizeUpgrade`. The owner can upgrade the implementation to arbitrary code without delay or governance oversight.
- **No emergency controls:** The contract has no pause mechanism or circuit-breaker functionality. The only way to halt operations is to remove all addresses from the whitelist individually or upgrade the implementation.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] The repayWithTokens() differs from the Pool's implementation

File(s): [src/WhitelistGateway.sol](#)

Description: The `WhitelistGateway` contract provides a `repayWithTokens(...)` function intended to allow users to settle their debts using their collateral tokens (`aTokens`). In the underlying `Pool` contract, the `repayWithTokens(...)` function operates by burning the user's `aTokens` directly to offset the debt. However, the `WhitelistGateway` does not call this function. Instead, it manually transfers `aTokens` from the user to itself, withdraws the underlying asset from the `Pool`, and then calls the standard `repay(...)` function.

This deviation in implementation introduces several issues. First, the `repayWithTokens(...)` function in the `Pool` contract is restricted by an `onlyWhitelistGateway` modifier, meaning it is effectively unreachable because the gateway never triggers it. Second, users expecting the gas-efficient or direct-burn behavior of the `Pool` logic will instead experience a multi-step process.

Most significantly, the gateway's approach can cause transactions to revert for users with a low health factor. When the gateway transfers `aTokens` from the user to itself, the user's collateral balance decreases before the debt is settled. This temporary drop in collateral can trigger a health factor violation in the `Pool` contract's `finalizeTransfer(...)` logic, causing the entire transaction to fail. In contrast, the `Pool` contract's native implementation avoids this by burning the tokens and reducing debt in a way that preserves or improves the health factor throughout the state change.

Recommendation(s): This difference in functionality can't be easily resolved. Calling `Pool.repayWithTokens()` in `WhitelistGateway.repayWithTokens()` would require significant changes to `Pool`'s code, which is not recommended. Alternatively, the `Pool.repayWithTokens()` could be called directly by users, with an additional modifier which would call `WhitelistGateway` to check if the user is whitelisted. If however this functionality should remain as is currently, we recommend to rename and document `WhitelistGateway.repayWithTokens()` function to inform users about different mechanisms.

Status: Acknowledged.

Update from the client: This difference will be documented.

6.2 [Low] Incorrect check order in the repayWithPermit(...) function

File(s): [src/WhitelistGateway.sol](#)

Description: The `repayWithPermit(...)` function attempts to execute a `permit(...)` call within a try-catch block. If the `permit(...)` fails, the catch block attempts a standard repayment. However, the catch block validates the current allowance against the original amount parameter rather than the actual `paybackAmount`.

If a user's debt is less than the amount passed to the function, the `paybackAmount` will be capped to the actual debt. However, if the `permit(...)` fails and the user's existing allowance is sufficient for the `paybackAmount` but less than the original amount, the transaction will revert unnecessarily.

Recommendation(s): Consider checking the allowance against the `paybackAmount` instead of the amount parameter within the catch block.

Status: Fixed.

Update from the client: Resolved in [fc93f800cf8f9bf76035d3337759ee4aad48514d](#).

6.3 [Info] Inaccurate NatSpec references regarding credit delegation and onBehalfOf

File(s): [src/WhitelistGateway.sol](#)

Description: The `NatSpec` documentation for the `borrow(...)` and `flashLoan(...)` functions suggests that the `onBehalfOf` parameter can be different from the `msg.sender`, implying support for credit delegation. However, the current implementation enforces that `msg.sender == onBehalfOf` and reverts with `DelegatedCallsNotAllowed` otherwise. This makes the existing documentation misleading as it describes functionality that has been intentionally restricted.

Recommendation(s): Consider updating the `NatSpec` for these functions to state that `onBehalfOf` must equal `msg.sender` and remove any outdated references to credit delegation.

Status: Fixed.

Update from the client: Resolved at [b64268253de9f0d01040cc193f7a2c851b85d131](#).

6.4 [Info] Missing token rescue functionality for tokens stuck in the gateway

File(s): [src/WhitelistGateway.sol](#)

Description: The `WhitelistGateway` contract lacks a mechanism to recover tokens that are accidentally sent to the contract address. Any ERC20 tokens or native assets sent to the gateway outside of the standard operational flow will be permanently locked unless the contract is upgraded to include a withdrawal function.

Recommendation(s): Consider implementing an `emergencyTokenTransfer(...)` function, restricted to the contract owner, that allows for the rescue of any tokens held by the contract. This is a standard pattern in Aave gateway contracts (see [WETHGateway.emergencyTokenTransfer](#)). The function should accept a token address, amount, and recipient as parameters.

Status: Fixed.

Update from the client: Resolved at [57fa601264dfb9c0c87f38c0e54b8e4c6782a24a](#).

6.5 [Info] POOL state variable has no public getter, reducing on-chain transparency

File(s): [src/WhitelistGateway.sol](#)

Description: The POOL state variable is declared with `internal` visibility, and the contract does not provide a corresponding public getter function.

```
/// @notice permissioned lending pool address
IPool internal POOL;
```

This prevents frontends, monitoring tools, and users from easily verifying which pool the gateway is currently targeting through a standard view call. While the `setPool(...)` function emits a `PoolUpdated` event, retrieving the current address requires scanning historical logs or inspecting raw storage slots, which reduces on-chain transparency.

Recommendation(s): Consider changing the visibility of the POOL variable to public or adding an explicit `getPool(...)` view function.

Status: Fixed.

Update from the client: Resolved at [bc7b7c697b282fe807030cd0ad677791732b8e](#).

6.6 [Info] POOL variable not set during initialization, creating non-functional deployment window

File(s): [src/WhitelistGateway.sol](#)

Description: The `initialize(...)` function initializes the contract's ownership and upgradeability but does not set the POOL address.

```
/// @notice Initializes the proxy with the owner address
function initialize(address owner) external initializer {
    __Ownable_init(owner);
    __UUPSUpgradeable_init();
}
```

As a result, the owner must call `setPool(...)` in a separate transaction after deployment. During the interval between these two transactions, the gateway is in a non-functional state. Any calls to core functions like `supply(...)`, `borrow(...)`, or `repay(...)` will revert because the POOL variable defaults to `address(0)`.

Recommendation(s): Consider updating the `initialize(...)` function to accept a pool address as a parameter to ensure the contract is fully functional immediately upon deployment.

Status: Acknowledged.

Update from the client: Acknowledged, we have cyclic dependency as we set the gateway in the pool contract on deployment.

6.7 [Info] The borrow(...) function always requires prior credit delegation to the gateway contract

File(s): [src/WhitelistGateway.sol](#)

Description: Every borrowing operation initiated through the WhitelistGateway requires the user to have previously granted a credit delegation allowance to the gateway via the `approveDelegation(...)` function on the `VariableDebtToken` contract.

When a user calls `borrow(...)` on the WhitelistGateway, the gateway forwards the request to the Pool contract. In this context, the `msg.sender` seen by the Pool is the gateway address, while the `onBehalfOf` parameter is the actual user's address. The `VariableDebtToken` contract's `mint(...)` function enforces a check to see if the caller and the beneficiary are the same:

```
function mint(address user, address onBehalfOf, uint256 amount, uint256 index)
    external virtual override onlyPool returns (bool, uint256)
{
    // @audit-issue Since user is the gateway and onBehalfOf is the user, this is always true.
    if (user != onBehalfOf) {
        _decreaseBorrowAllowance(onBehalfOf, user, amount);
    }
    return (_mintScaled(user, onBehalfOf, amount, index), scaledTotalSupply());
}
```


Because the `WhitelistGateway` acts as an intermediary, `user != onBehalfOf` will always evaluate to true. This triggers the `_decreaseBorrowAllowance(...)` function in the `DebtTokenBase` contract:

```
function _decreaseBorrowAllowance(address delegator, address delegatee, uint256 amount)
    internal
{
    uint256 allowance = _borrowAllowances[delegator][delegatee];
    // ...
    // @audit-issue This subtraction reverts with Panic(0x11) if allowance is 0.
    _borrowAllowances[delegator][delegatee] = allowance - amount;
    // ...
}
```

Given the protocol treats any borrowing as if the gateway is borrowing on the user's credit, then an explicit allowance becomes necessary.

Recommendation(s): This behavior should be properly documented and clear to users.

Status: Fixed.

Update from the client: Resolved at [a644ffcb5ea6edef8aef0bcc6807c72e8090e02b](#).

6.8 [Info] The `repayWithPermit(...)` function hardcodes the interest rate mode

File(s): `src/WhitelistGateway.sol`

Description: Unlike the standard `repay(...)` function, which allows users to specify an `interestRateMode`, the `repayWithPermit(...)` function hardcodes the use of `DataTypes.InterestRateMode.VARIABLE`. This limitation is not documented in the function's NatSpec. Additionally, the function calls `permit(...)` using the original amount provided by the user, even if the actual debt (the `paybackAmount`) is lower. This can leave a residual allowance granted to the gateway if the user permits more than what is eventually consumed.

Recommendation(s): Consider updating the NatSpec for `repayWithPermit(...)` to clarify that it only supports variable interest rate debt and noting the potential for residual allowances.

Status: Fixed.

Update from the client: Resolved at [a644ffcb5ea6edef8aef0bcc6807c72e8090e02b](#).

6.9 [Info] The gateway does not wrap `setUserUseReserveAsCollateral(...)` and `setUserEMode(...)`

File(s): `src/WhitelistGateway.sol`

Description: The `WhitelistGateway` is documented as the entry point for whitelisted users to perform all core operations. However, it does not provide wrappers for `setUserUseReserveAsCollateral(...)` and `setUserEMode(...)`. While users can still call these functions directly on the `Pool` contract (as they are not restricted by the `onlyWhitelistGateway` modifier), this creates an inconsistent user experience where most actions go through the gateway but specific configuration changes require direct pool interaction.

Recommendation(s): Consider adding wrapper functions for these operations to the `WhitelistGateway` to provide a complete interface for whitelisted users.

Status: Acknowledged.

Update from the client: Acknowledged, these are self configuration functions which do not affect funds movement and there is no `onBehalfOf` parameter support.

6.10 [Best Practice] The number of gap slots should sum up with other variables to a round number

File(s): `WhitelistGateway.sol`

Description: The `WhitelistGateway` contract defines a `__gap` array of size 50. When combined with the existing state variables, the total number of storage slots occupied is 52. Standard best practices according to [OpenZeppelin](#) suggests that the total number of slots (variables plus gap) should equal a round number, such as 50, to simplify storage management during future upgrades.

The `__gap` is set up to 50. This sets a total number of slots taken to 52. The [OpenZeppelin's recommendation](#) states that the sum of taken slots by variables and `__gap` should sum up to a round number, e.g. 50.

Recommendation(s): Consider adjusting the `__gap` array size so that the total number of occupied slots in the contract equals a round number.

Status: Fixed.

Update from the client: Resolved at [9c05421514130c18d79faf2a75e51f9436cb2d9e](#).

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- **Technical whitepaper:** A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- **User manual:** A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- **Code documentation:** Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- **API documentation:** API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- **Testing documentation:** Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- **Audit documentation:** Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Hyperlend's documentation

The Hyperlend's WhitelistGateway contract has extensive natSpec documenting the code.

8 Test Suite Evaluation

8.1 Tests Output

```
Ran 21 tests for tests/WhitelistGatewayFork.t.sol:WhitelistGatewayForkTest
[PASS] testFuzz_borrow(uint96,bool) (runs: 1000, : 724316, ~: 720292)
[PASS] testFuzz_deposit(uint96,bool) (runs: 1000, : 406110, ~: 414998)
[PASS] testFuzz_flashLoan(uint96,uint96) (runs: 1000, : 1451061, ~: 1452300)
[PASS] testFuzz_flashLoanSimple(uint96,bool) (runs: 1000, : 856744, ~: 854309)
[PASS] testFuzz_liquidationCall(uint96) (runs: 1000, : 1164044, ~: 1164066)
[PASS] testFuzz_liquidationCall_doesNotDrainGatewayCollateral(bool) (runs: 1000, : 1478268, ~: 1529330)
[PASS] testFuzz_liquidationCall_receiveAToken(uint96) (runs: 1000, : 1277589, ~: 1277620)
[PASS] testFuzz_repay(uint96,bool) (runs: 1000, : 830293, ~: 830182)
[PASS] testFuzz_repayWithATokens(uint96,bool) (runs: 1000, : 1250067, ~: 1253869)
[PASS] testFuzz_repayWithPermit(uint96) (runs: 1000, : 859078, ~: 859103)
[PASS] testFuzz_supply(uint96,bool) (runs: 1000, : 405990, ~: 396749)
[PASS] testFuzz_supplyWithPermit(uint96) (runs: 1000, : 447968, ~: 447988)
[PASS] testFuzz_withdraw(uint96,uint96,bool,bool) (runs: 1000, : 570216, ~: 570236)
[PASS] test_addAndRemoveWhitelist() (gas: 1918683)
[PASS] test_borrow_revertsWhenOnBehalfOfIsNotSender() (gas: 576453)
[PASS] test_deposit_revertsWhenCallerIsNotWhitelistGateway() (gas: 216725)
[PASS] test_deposit_revertsWhenNotWhitelisted() (gas: 203800)
[PASS] test_flashLoan_revertsWhenOnBehalfOfIsNotSender() (gas: 407217)
[PASS] test_repayWithPermit_WhenPermitFrontRunHappens() (gas: 863845)
[PASS] test_supplyWithPermit_succeedsAfterPermitFrontRun() (gas: 461139)
[PASS] test_withdraw_doesNotDrainGatewayATokens() (gas: 751797)
Suite result: ok. 21 passed; 0 failed; 0 skipped; finished in 8.70s (35.22s CPU time)

Ran 1 test suite in 8.71s (8.70s CPU time): 21 tests passed, 0 failed, 0 skipped (21 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.